AMSS 2026 — Lab 3: Critique Session — Structural Artifacts

Traian-Florin Șerbănuță

2026

Lab 3: Red-Team the Structure

Three phases, 100 minutes — **you work in teams of 3-5.**

1. **Brief + rubric** (~10 min) — the domain, the three artifacts, the read-order.
2. **Defect hunt** (~60 min) — your team hunts every planted defect in three flawed artifacts, rating each by severity.
3. **Defect-hunt-off** (~30 min) — reveal ground truth, score, crown the sharpest critics.

Deliverable: a **severity-rated defect log** per team, committed to the course lab repo.

The Flip: From Driver to Red Team

- **Lab 2:** you drove AI to a class diagram, then critiqued *your own* output.
- **Today:** we prepared three flawed artifacts. Your job is to **find what's wrong** — and how badly.

No AI to drive this lab. Pure reading and critique — the architect-and-critic loop, all critic.

Before You Start

- Form teams of **3-5**. One member is the **scribe** (owns the defect log + the commit).
- You receive at lab start: your **team-id**, the lab-repo URL, and the three artifacts (in `lab03/README.md` on clone).
- All three artifacts model the **same** system — the airport parking lot below.

The Domain: Airport Parking Lot

The artifacts model an airport parking lot. The **domain is honest** — every defect is in the artifacts, not the description.

- Multiple **levels**; each level has many **parking spots**, each with a free/occupied sensor.
- Multiple **entrances** and **exits**, each with a plate-reading **camera**.
- The **entry barrier** prints a **ticket**; the **exit barrier** opens only if the ticket is paid.
- **Payment kiosks** on each level; payment is by **cash** or **card**.
- **Display boards** at entrances show free spots, total and per level.

The Three Artifacts

All three describe the same parking lot, and **all three are flawed**:

1. A **class diagram** — the structure of the domain types.
2. A **package diagram** — how the system is grouped.
3. A **component diagram** — how it decomposes into replaceable units.

Find every planted defect. Rate each. The team that scores highest wins the hunt-off.

Your Rubric: the Read-Orders

Class diagram (Week 4):

1. Real **domain** classes? (or invented infrastructure / god class)
2. **Multiplicities** right? (read each aloud)
3. Each **association** names a real verb?
4. **Whole-part** relationships captured? (aggregation / composition)

Package + component (Week 5):

5. Right **grain**? (not one box; not a box per class)
6. **Dependencies** one way? (no cycles, layering respected)
7. Each **component** defined by its interfaces?

The Defect Vocabulary

Name each defect with the catalogue from Weeks 4-5:

- **Class:** wrong multiplicity · fake association · missing aggregation · invented class/infrastructure · god class · is-a flattened to an attribute.
- **Package/component:** under-decomposition · over-decomposition · invented infrastructure · dependency cycle / wrong direction · component with no interfaces.

A defect named with the catalogue scores; a vague "this looks off" does not.

Severity — Rate Every Defect

- **High (3)** — breaks the design or propagates downstream: a god class, a dependency cycle, a wrong core multiplicity.
- **Medium (2)** — wrong but local: invented infrastructure, a mis-grouped class, is-a as an attribute.
- **Low (1)** — cosmetic or clarity: a fake decorative line, a vague package name.

Every logged defect needs a severity. Rating *is* the skill — a critic who can't prioritise drowns the real problems.

Phase 2: The Defect Hunt (60 min)

As a team, walk each artifact with the read-order. For **every** defect, log:

- **Artifact** (class / package / component)
- **Defect name** (from the catalogue) + **where** (the class, line, or dependency)
- **Severity** (high / med / low)
- **One line: why it matters**

Divide and conquer, or sweep together — but converge on one shared log.

How the Hunt-Off Is Scored

Calibrate — precision counts:

- **Correct defect** scores its severity weight (high 3 / med 2 / low 1).
- **Severity** must be within one band for full credit; off by two bands loses a point.
- **False positive** — a "defect" that isn't real — costs a point (minus 1).
- No double-counting the same defect under two names.

Spraying guesses backfires. Find the real defects, rate them honestly.

The Defect Log (Deliverable)

One table, all three artifacts, on branch `lab03/<team-id>`:

Artifact	Defect (catalogue name)	Where	Severity	Why it matters
class	wrong multiplicity	`ParkingLot 1--1 Level`	high	a lot has many levels
package	dependency cycle	payments <-> barriers	high	neither builds alone
component	no interfaces	`PaymentComponent`	high	just a renamed class

Phase 3: Defect-Hunt-Off + Submit (30 min)

- We reveal ground truth **artifact by artifact**; each team scores its own log live.
- Each team nominates its **best catch** (highest-severity real defect) — bonus point.
- Highest net score wins.

```
git checkout -b lab03/<team-id>
mkdir -p lab03/<team-id>
git add lab03/<team-id>/defect-log.md
git commit -m "Lab 3: <team-id> parking-lot defect log"
git push -u origin lab03/<team-id>
```

Grading — Pass / Redo

Pass needs both:

1. `defect-log.md` committed by the team by the deadline.
2. The log names **at least five** real defects across the three artifacts, each with a severity and a one-line reason, **including at least one high**.

The competition is for sharpness and bragging rights; the gate is an honest, catalogue-named log. We grade your critique, not your speed.

Why This Matters

Red-teaming AI's structural output — naming defects and rating them cold — is exactly the **Critique** skill the oral defense checks. A flawed structure does not stay contained: it propagates to behaviour, tests, and code.

Next: **Week 7** widens behaviour to state + activity; **Lab 4** is this same defect hunt, on **behavioural** artifacts (sequence, state, activity).